



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-5223 Crates in third party registries can override the cached source of other crates

Vulnerability Report

Classification	TLP:CLEAR
Published	2026-06-14
Version	1.0
Author	Lost Boys Cyber
Report Type	Vulnerability Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-5223 Crates in third party registries can override the cached source of other crates - Vulnerability Report

Published: 2026-06-14 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Intelligence Requirement
3. Source Review and Confidence
4. Key Findings
5. Threat Context
6. Detection and Hunting
7. Recommendations
8. References

1. Executive Summary

CVE-2026-5223 is a medium-severity Cargo supply-chain integrity flaw affecting users of third-party Rust crate registries. Rust upstream states that Cargo incorrectly handled symlinks inside crate tarballs downloaded from third-party registries, allowing a malicious crate to overwrite the cached source of another crate from the same registry. In practical terms, a poisoned package can tamper with a downstream dependency's extracted source before a build consumes it.

The exposure is narrower than an ecosystem-wide crates.io compromise, but the defensive implications are still material for organizations that rely on private, partner-hosted, or multi-tenant alternate registries. Rust upstream explicitly says `crates.io` users are not affected because crates.io forbids uploads containing symlinks. The main risk therefore concentrates in software factories, internal developer platforms, CI/CD pipelines, and partner ecosystems that permit third-party registry usage and do not independently reject symlinks at the registry layer.

This flaw is best treated as a dependency-integrity and cache-poisoning problem rather than a classic malware IOC problem. A successful exploit can cause unexpected code to be compiled from the local Cargo cache, opening a path to supply-chain compromise, malicious build outcomes, and difficult-to-trace trust failures inside engineering workflows. The most important immediate actions are to upgrade to Rust 1.96.0 or later, verify whether alternate registries are in use, and audit registry content and policy for symlink handling.

Detection and hunting should prioritize exposure discovery, registry governance, and suspicious Cargo workflows rather than waiting for public attacker indicators. Defenders should inventory vulnerable Cargo versions, locate alternate-registry use in developer and build environments, review registry or proxy logs for suspicious crate pulls, and investigate whether unexpected symlink-bearing tarballs or unusual source-cache modifications can be observed around Cargo activity.

2. Intelligence Requirement

This report addresses the following intelligence requirement: what does CVE-2026-5223 mean for defenders that use third-party Cargo registries, which environments are materially exposed, and what remediation, detection, and hunting steps should be prioritized?

Question	Assessment
What is the flaw?	Cargo accepted symlinks inside crate tarballs from third-party registries and could let one malicious crate overwrite the cached source of another crate from the same registry.
What is the main impact?	Local Cargo source-cache poisoning that can alter dependency source code prior to build or compilation.
Who is exposed?	Organizations using third-party or private Cargo registries, especially shared or multi-tenant registries and CI/CD systems consuming those registries.
Who is not exposed?	Rust upstream states that users of `crates.io` are not affected because crates.io forbids symlink-bearing crate uploads.
Which versions are affected?	All Cargo versions shipped before Rust 1.96.0.
What fixes the issue?	Rust 1.96.0 and later, which reject extraction of any symlink within crate tarballs.
Is exploitation broadly reported?	Reviewed public sources did not identify authoritative evidence of widespread in-the-wild exploitation, CISA KEV inclusion, or broad campaign reporting at assessment time.
What deserves highest priority?	Alternate-registry operators, build pipeline owners, developer-platform teams, and any environment where registry trust

boundaries extend beyond crates.io.

3. Source Review and Confidence

Source	Contribution	Confidence
Rust security advisory blog post for CVE-2026-5223	Canonical flaw description, affected versions, mitigation guidance, and crates.io carve-out	High
Rust 1.96.0 release notes	Confirms fix availability and summarizes both registry-related Cargo CVEs fixed in the release	High
Cargo Book: Registries	Documents alternate-registry support and registry credential/provider behavior relevant to exposure triage	High
Cargo Book: Registry Index	Explains registry index mechanics and sparse-registry behavior relevant to how third-party registries differ operationally from crates.io	High
Cargo Book: Registry Authentication	Explains token and credential-provider handling that shapes operational blast radius in build environments	High
OpenCVE / CNA mirrored CVE record	Provides CVSS v4 6.5, CVSS v3.1 5.3, CWE-61 classification, patch reference, and the affected version range ` <code>< 1.96.0</code> `	Medium
Microsoft Security Update Guide syndication entry	Workflow trigger confirming public publication of the CVE in the intake feed	Medium

Confidence is high for the technical core of this assessment because the upstream Rust advisory, Rust 1.96.0 release notes, and the mirrored CNA/OpenCVE record align on the flaw mechanics, affected-version ceiling, and remediation path. Confidence is lower for exploitation prevalence because the reviewed public material focused on disclosure and patching rather than incident reporting, intrusion evidence, or public proof-of-compromise case studies.

Two analytical caveats matter. First, the vulnerability is meaningfully narrower than a crates.io-wide software ecosystem event; the public advisory explicitly limits the impact to third-party registries because crates.io forbids symlink-bearing packages. Second, the downstream business impact is environment-specific: some teams use alternate registries only for low-risk mirrors, while others rely on private registries for sensitive internal dependencies, release pipelines, and privileged build systems where cache poisoning can have outsized security consequences.

4.1 Vulnerability metadata

Field	Value
CVE	CVE-2026-5223
Product family	Cargo / Rust toolchain
Vulnerability class	Source-cache poisoning via symlink handling in third-party registry crate tarballs
Primary weakness	CWE-61: UNIX Symbolic Link (Symlink) Following

Vendor severity	Medium
Public scores	CVSS v4 6.5 / 10; CVSS v3.1 5.3 / 10
Affected versions	Cargo versions shipped before Rust 1.96.0
Fixed version	Rust 1.96.0 / Cargo build that rejects symlink extraction from crate tarballs
Public disclosure	2026-05-25 upstream Rust advisory; 2026-06-13 MSRC syndication entry reviewed by this workflow
Exploitation status	No authoritative broad in-the-wild exploitation reporting identified during this review
Notable carve-out	`crates.io` users are not affected according to Rust upstream
Patch reference	rust-lang/cargo pull request `#17031`

4.2 Flaw mechanics

Rust's advisory explains that Cargo extracts downloaded crate source into a local cache under `~/cargo` and reuses that cache across future builds. Cargo includes protections intended to stop archive members from escaping a crate's own extraction directory. The defect is that a malicious crate tarball from a third-party registry can use symlinks to write one level below the crate's expected cache directory, which in Cargo's cache layout can overwrite another crate's cached source within the same registry namespace.

This matters because dependency compilation normally assumes that the cached source of a crate faithfully reflects what the registry served for that crate. CVE-2026-5223 breaks that assumption. An attacker who can get a victim to consume a malicious crate from an affected third-party registry can potentially modify the local cached source tree of another dependency before the victim builds or audits it. That makes the flaw a cache-poisoning and dependency-integrity issue rather than a direct remote-code-execution primitive in Cargo itself.

Rust also notes that the practical mitigation impact is low for legitimate publishers because `cargo package` and `cargo publish` do not add symlinks. The defensive problem therefore centers on registry-side acceptance of malicious uploads and Cargo-side trust in extracted archive contents from non-crates.io sources.

4.3 Exposure profile

Exposure pattern	Why it matters	Priority
Private or partner-hosted Cargo registries	Directly matches the third-party-registry precondition in the advisory	Immediate
Multi-tenant registry platforms	A malicious tenant may be able to publish a crate that poisons another crate's cached source for shared consumers	Immediate
CI/CD runners building from alternate registries	Build systems often consume dependencies automatically and may produce signed or production artifacts from poisoned source	High
Developer workstations using authenticated alternate registries	Local builds can consume tampered cache contents and mask the root cause as an ordinary dependency issue	High
Registry implementations that already reject symlink-bearing tarballs	Reduce exploitability materially even before Cargo is upgraded	Medium

`crates.io`-only environments	Upstream says these are not affected by this CVE	Reduced
-------------------------------	--	---------

4.4 Defensive significance and ATT&CK framing

MITRE ATT&CK mapping for CVE-2026-5223 is conditional and should be read as defensive context rather than proof of a known actor workflow. The vulnerability enables dependency and cache tampering inside software supply chains, not a specific malware family or campaign.

ATT&CK ID	Technique	Rationale
T1195.001	Supply Chain Compromise: Compromise Software Dependencies and Development Tools	Exploitation requires attacker-controlled package content inside a software dependency workflow.
T1036	Masquerading	Overwritten cache contents can cause one crate's source to appear to be another crate's trusted code during build review or compilation.
T1071.001	Application Layer Protocol: Web Protocols	Cargo retrieves registry content over web protocols, making registry and proxy telemetry relevant to exposure hunting.

The practical takeaway is that alternate registries should be treated as trust-boundary extensions of the build system. If a registry accepts malicious tarballs, the resulting compromise can propagate into local source caches, automated builds, and downstream software artifacts without relying on a traditional endpoint exploit chain.

5. Threat Context

CVE-2026-5223 sits at the intersection of dependency management, archive handling, and software supply-chain trust. It does not create a universal internet worm or a broad host-exploitation condition. Instead, it weakens the assumption that cached dependency source pulled from a trusted registry remains isolated from other crates during extraction. In organizations where alternate registries are central to internal package distribution, that assumption is foundational.

The most relevant threat model is a malicious or compromised publisher operating within a third-party registry ecosystem. If the attacker can get a victim environment to download the malicious crate, cache poisoning can alter a second crate's local source prior to build, code review, or testing. That can undermine reproducibility, inject unauthorized logic into compiled artifacts, or create subtle downstream build drift that is difficult to diagnose after the fact.

5.1 Representative attack scenarios

Scenario	Description	Defensive meaning
Malicious publisher on a shared private registry	Attacker uploads a crate containing symlink entries that overwrite another crate's cached source when a victim resolves dependencies	Review alternate-registry trust, publisher controls, and upload validation policies
Compromised partner registry	A partner-hosted registry serves poisoned tarballs to build infrastructure that assumes Cargo cache isolation	Prioritize registry trust review, package provenance checks, and build-tier patching
CI/CD dependency poisoning	Shared runners consume a malicious dependency from a third-party registry and compile from altered cached source into release artifacts	Treat build runners and signing pipelines as high-priority remediation targets
Developer workstation source drift	Local dependency builds use a poisoned	Hunt for alternate-registry use, unexpected

	cache, causing confusing code mismatches or unauthorized behavior during testing	cache changes, and suspicious Cargo workflows
--	--	---

5.2 Exploitation-status assessment

Reviewed public sources did not identify authoritative evidence of widespread exploitation, CISA Known Exploited Vulnerabilities catalog inclusion, or campaign reporting tied to CVE-2026-5223 at assessment time. The available material instead points to a newly disclosed supply-chain weakness with moderate severity and a clear vendor remediation path.

That absence of public exploitation reporting should not be treated as a reason to defer action in exposed environments. Build systems, developer platforms, and internal registries carry disproportionate trust. A medium-severity flaw that can alter dependency source code may be strategically more important to a mature engineering organization than a noisier internet-facing CVE with less leverage over signed software production.

6. Detection and Hunting

Detection for CVE-2026-5223 should emphasize exposure discovery, build-workflow visibility, and signs of suspicious registry-delivered content rather than waiting for a stable attacker IOC set. Reviewed public sources did not identify public hashes, domains, or campaign-specific infrastructure. This is therefore a software supply-chain and package-integrity hunting problem.

6.1 Detection coverage summary

Signal	Telemetry source	Analytic goal	Coverage caveat
Cargo versions below 1.96.0	Software inventory, SBOM, developer-tool telemetry	Identify vulnerable toolchains	Distro or vendor backports may remediate without obvious version parity
Alternate-registry use in Cargo configs or environment variables	Repo scanning, <code>`.cargo/config.toml`</code> , CI secrets/config, process logs	Find environments that meet the third-party-registry prerequisite	Some registry definitions are inherited from templates or injected at runtime
Cargo extracting or building dependencies from non-default registries	Process creation telemetry, CI logs, EDR command lines	Prioritize systems where a poisoned crate could influence builds	Process telemetry alone does not prove malicious tarball content
Archive or file activity in Cargo cache paths after dependency resolution	Endpoint file telemetry, host auditing, controlled lab traces	Surface suspicious writes under <code>~/cargo/registry/src/`</code>	File-write visibility varies by platform and can be noisy on developer endpoints
Registry or proxy requests for unusual crate artifacts	Reverse-proxy logs, registry access logs, artifact repository telemetry	Detect suspicious third-party registry package pulls and follow-up investigation targets	Most logs show package pulls but not whether tarballs contained symlinks

6.2 Sigma - Cargo third-party registry usage requiring CVE-2026-5223 exposure review

```

title: Cargo Third-Party Registry Usage Requiring CVE-2026-5223 Exposure Review
id: e1eac028-15f9-4d47-9e27-c6fc6f1bf644
status: experimental
description: Detects Cargo executions associated with alternate registries or dependency build activity that may warrant review for CVE-2026-5223 source-cache poisoning exposure.
author: Lost Boys Cyber
date: 2026-06-14
references:
  - https://blog.rust-lang.org/2026/05/25/cve-2026-5223/
  - https://blog.rust-lang.org/2026/05/28/Rust-1.96.0/
logsource:
  category: process_creation
  product: windows

```

```

detection:
  image:
    Image|endswith:
      - '\\cargo.exe'
  commands:
    CommandLine|contains:
      - ' build '
      - ' test '
      - ' fetch '
      - ' install '
      - ' publish '
      - ' --registry '
      - ' sparse+https://'
      - ' CARGO_REGISTRIES_'
  condition: image and commands
falsepositives:
  - Legitimate developer or CI workflows using private registries
  - Security-team validation of Cargo registry controls
level: medium
tags:
  - attack.t1071.001
  - attack.t1195.001

```

Linux and macOS estates should adapt image paths and field names to local schemas. This rule is intended to locate candidate exposure and investigation targets, not to claim direct exploit confirmation.

6.3 KQL - identify alternate-registry Cargo activity on build and developer systems

```

let lookback = 30d;
DeviceProcessEvents
| where Timestamp > ago(lookback)
| where FileName in~ ("cargo", "cargo.exe")
| where ProcessCommandLine has_any (" build ", " test ", " fetch ", " install ", " publish ", " --
registry ", "CARGO_REGISTRIES_", "sparse+https://")
| project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine, InitiatingProcessFileName,
InitiatingProcessCommandLine
| order by Timestamp desc

```

6.4 KQL - review file activity in Cargo cache locations after dependency operations

```

let lookback = 30d;
DeviceFileEvents
| where Timestamp > ago(lookback)
| where FolderPath has '\\.cargo\\registry\\src\\' or FolderPath has './.cargo/registry/src/'
| where InitiatingProcessFileName in~ ("cargo", "cargo.exe")
| project Timestamp, DeviceName, ActionType, FolderPath, FileName, InitiatingProcessFileName,
InitiatingProcessCommandLine
| order by Timestamp desc

```

6.5 Hunt questions

Hypothesis	What to prove or disprove
Some engineering systems consume dependencies from third-party Cargo registries	Identify registry URLs, owners, tenant models, and whether crates.io is bypassed for sensitive packages
Vulnerable Cargo versions remain in CI/CD or developer environments	Map installed versions, backports, and runtime toolchain selection across builds
Registry upload controls permit symlink-bearing crate tarballs	Review registry policy, artifact scanning, and package-validation controls with platform owners
Cargo cache paths have shown unusual modification patterns after dependency fetch/build operations	Examine file telemetry and build logs around alternate-registry package pulls
Security teams currently lack visibility into alternate-registry	Use process and config hunting to build an exposure inventory

dependency use	before patch verification
----------------	---------------------------

No stable IOC set was identified from reviewed public sources. Hunting should center on registry trust, package validation, build-tier visibility, and vulnerable Cargo deployment discovery.

7. Recommendations

Priority	Owner	Action	Rationale
Immediate	Platform engineering / developer enablement	Upgrade Rust toolchains and Cargo to 1.96.0 or later everywhere alternate registries are used	Removes the vulnerable symlink-extraction behavior
Immediate	Registry owners	Audit third-party/private registries for symlink-bearing crates and enable or build upload rejection controls for symlink entries	Directly addresses the archive-content precondition described by Rust upstream
Immediate	CI/CD owners	Prioritize shared runners, release pipelines, and signing environments for upgrade validation	Build infrastructure has disproportionate supply-chain impact if poisoned source is compiled
High	Security engineering	Inventory all alternate-registry usage through config review, process telemetry, and repository scanning	Establishes the true exposure surface before remediation is declared complete
High	Registry / package-platform teams	Review tenant isolation, publisher trust, package-review controls, and provenance requirements for non-crates.io registries	Reduces the likelihood that a malicious publisher can weaponize the flaw
High	Detection engineering	Add hunts for Cargo alternate-registry activity and Cargo cache-path file changes tied to build operations	Improves visibility into affected workflows and suspicious follow-on behavior
Medium	Application security / build engineering	Validate reproducible-build assumptions and dependency provenance for sensitive Rust services	Helps detect unauthorized source drift introduced before patching
Medium	Vulnerability management	Confirm vendor, distro, or container backports before closing findings based only on version-string checks	Prevents false remediation conclusions in mixed packaging environments

Recommended execution order:

- Inventory all alternate-registry usage and identify which engineering systems trust third-party Cargo sources.
- Upgrade affected Cargo versions to Rust 1.96.0 or later and verify any distro or vendor backports.
- Review registry upload policy for symlink rejection and audit existing registry contents for symlink-bearing crates.
- Prioritize CI/CD runners, release builders, and high-trust developer platforms for validation and follow-on hunts.

- Add monitoring for alternate-registry Cargo activity and suspicious cache-path file events while remediation is being completed.

8. References

- Rust Blog, `Security Advisory for Cargo (CVE-2026-5223)` - <https://blog.rust-lang.org/2026/05/25/cve-2026-5223/>
- Rust Blog, `Announcing Rust 1.96.0` - <https://blog.rust-lang.org/2026/05/28/Rust-1.96.0/>
- Cargo Book, `Registries` - <https://doc.rust-lang.org/cargo/reference/registries.html>
- Cargo Book, `Registry Index` - <https://doc.rust-lang.org/cargo/reference/registry-index.html>
- Cargo Book, `Registry Authentication` - <https://doc.rust-lang.org/cargo/reference/registry-authentication.html>
- OpenCVE CVE record for CVE-2026-5223 - <https://app.opencve.io/cve/CVE-2026-5223>
- Microsoft Security Update Guide syndication entry - <https://msrc.microsoft.com/update-guide/vulnerability/CVE-2026-5223>